

Reinforcement Learning

Slide 2

REINFORCEMENT LEARNING

El Aprendizaje por Refuerzo (**RL**) es una técnica de inteligencia artificial en la que un agente aprende a tomar decisiones mediante prueba y error, recibiendo recompensas o penalizaciones según sus acciones.

Slide 3

ANALOGÍA

Es como aprender a jugar un videojuego: al principio cometemos muchos errores, pero con cada intento aprendemos qué acciones nos acercan más a obtener una buena puntuación.

Slide 4

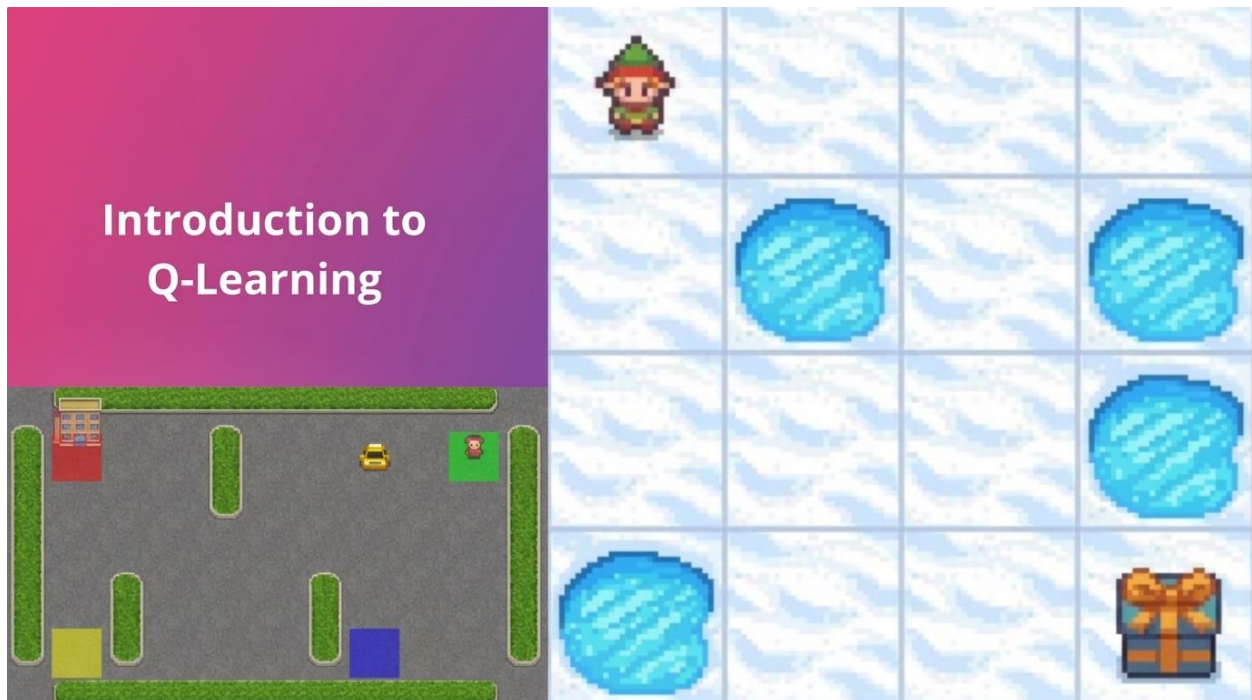


Figure 1 Introduction to Q-learning

Slide 5

Vamos a profundizar en uno de los métodos de aprendizaje por refuerzo, los métodos basados en el valor, y a estudiar un algoritmo de aprendizaje por refuerzo: el Q-Learning.

Slide 6

La idea de esta clase es profundizar en los siguientes temas:

- Métodos basados en valores.
- Diferencias entre el aprendizaje de Monte Carlo y el aprendizaje por diferencias temporales.
- Estudiaremos e implementaremos un algoritmo de aprendizaje por refuerzo (RL): el aprendizaje Q (Q-Learning).

Nota: El RL es una aproximación a lo que hoy conocemos como Game-playing agents.

Slide 7

En RL construimos agentes que toman decisiones inteligentes como . Por ejemplo un agente que juega videojuegos o invierte en la bolsa. Siempre buscando maximizar sus beneficios.



Figure 2 Agent Environment Relation

Para tomar decisiones inteligentes, nuestro agente aprenderá del entorno interactuando con él mediante el método de prueba y error y recibiendo recompensas (positivas o negativas) como retroalimentación.

Slide 8

El proceso de toma de decisiones del agente se denomina política π : dada una situación, una política generará una acción o una distribución de probabilidades sobre las acciones. Es decir, dada una observación del entorno, una política proporcionará una acción (o varias probabilidades para cada acción) que el agente debería llevar a cabo.

Slide 9



Figure 3 Policy

Slide 10

Nuestro objetivo es encontrar una política óptima π^* , es decir, una política que conduzca a la mejor recompensa acumulada esperada.

Y para encontrar esta política óptima (y, por lo tanto, resolver el problema de RL), existen dos tipos principales de métodos de RL:

- *Métodos basados en políticas*: Entrenan directamente la política para aprender qué acción tomar dado un estado.

- *Métodos basados en valores*: Entrenan una función de valor para aprender qué estado es más valioso y utilizan esta función de valor para tomar la acción que conduce a él.

Slide 11

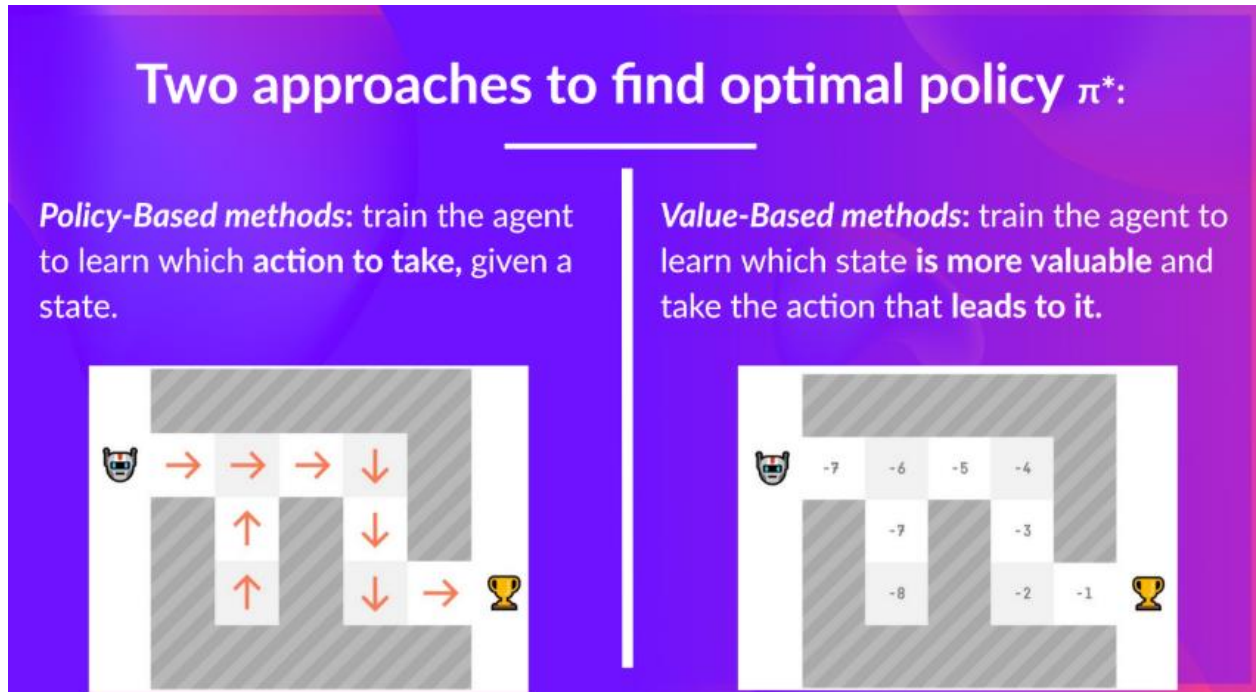


Figure 4 Approaches

Slide 12

El objetivo de un agente de RL es tener una política óptima π^* .

Para encontrar la política óptima, tenemos 2 dos métodos diferentes:

Slide 13

MÉTODOS BASADOS EN POLÍTICA

Entrenamos directamente la política para seleccionar qué acción tomar dado un estado (o una distribución de probabilidad sobre las acciones en ese estado).

En este caso, no tenemos una función de valor. La política recibe un estado como entrada y devuelve qué acción tomar en ese estado

Nota: Una política determinista es una política que devuelve una única acción dado un estado, a diferencia de una política estocástica, que devuelve una distribución de probabilidad sobre las acciones.

Slide 14

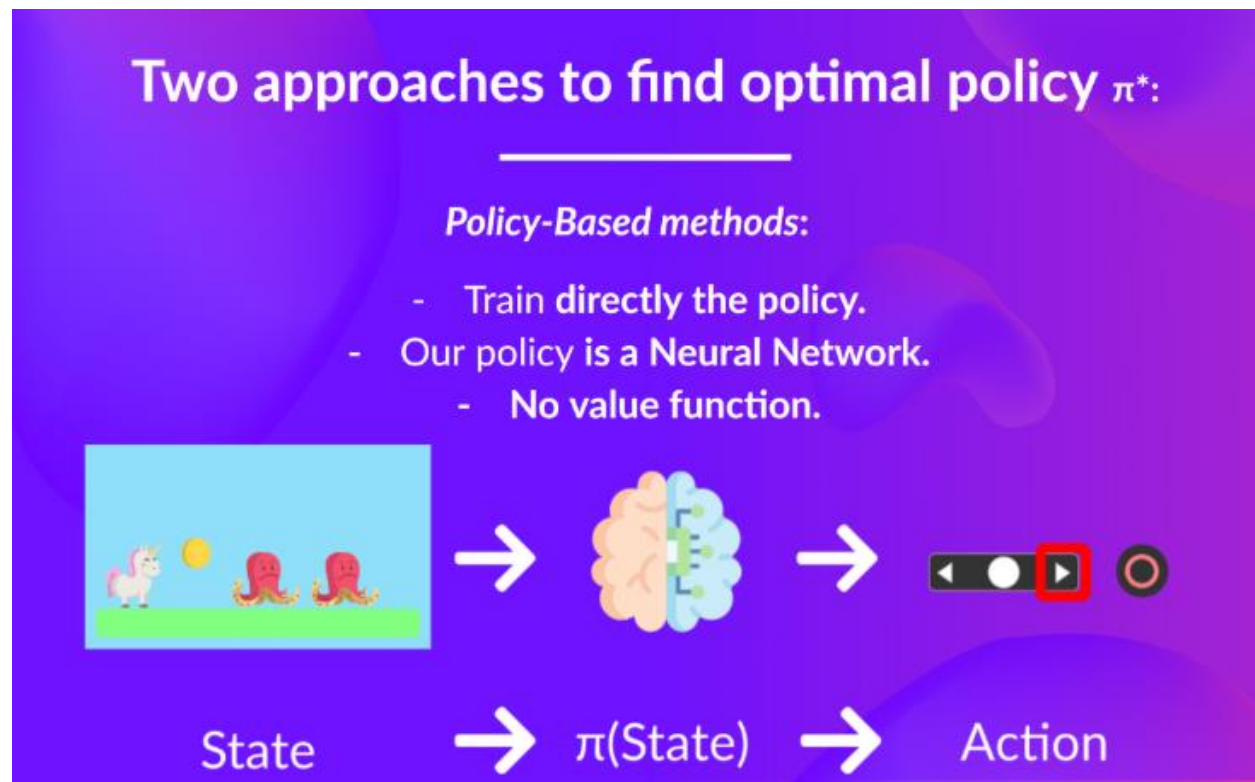


Figure 5 Policy-Based Methods

No definimos manualmente el comportamiento de nuestra política; es el entrenamiento el que lo definirá.

Slide 15

Métodos basados en valores

De forma indirecta, entrenando una función de valor que devuelve el valor de un estado o de un par estado-acción. Dada esta función de valor, nuestra política tomará una acción.

Como la política no se entrena ni se aprende, tenemos que especificar su comportamiento. Por ejemplo, si queremos una política que, dada la función de valor, tome acciones que siempre conduzcan a la mayor recompensa, crearemos una Política Greedy (codiciosa).

Nota: *greedy* significa que la política elige en cada momento la acción que parece mejor según la función de valor

Slide 16

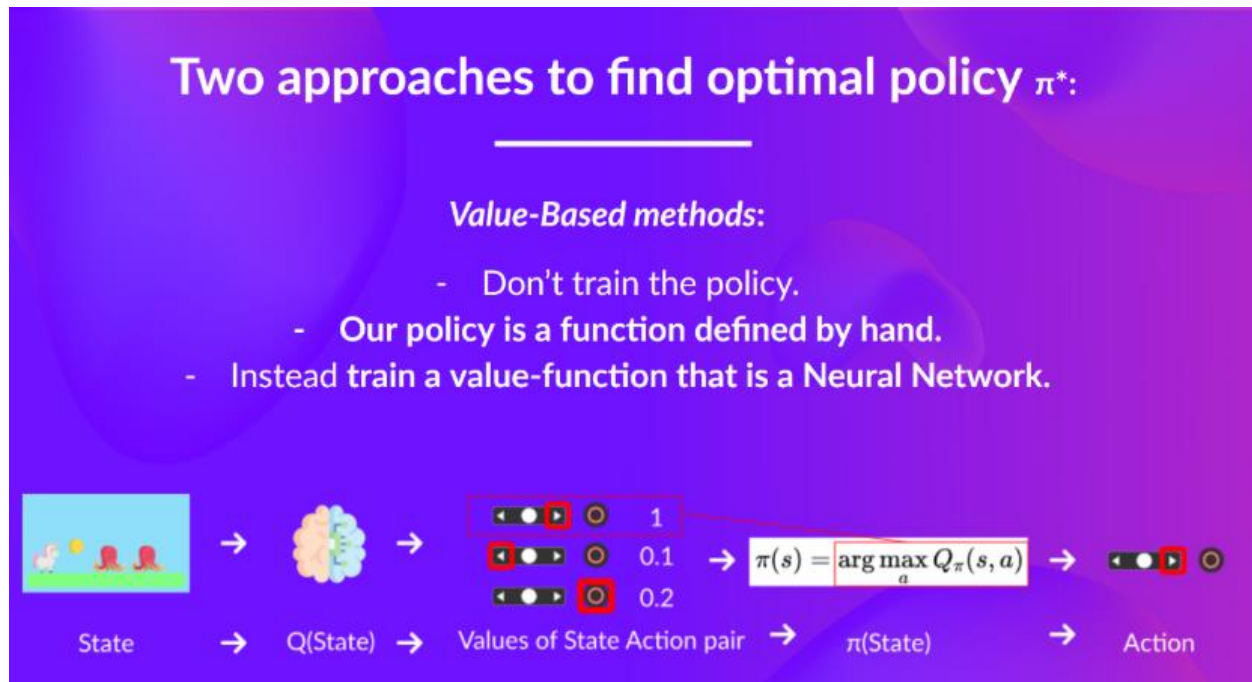


Figure 6 Value-Based Method

Dado un estado, nuestra función action-value (la cual entrenamos), generara el valor de cada action en ese estado, luego nuestra política «greedy» selecciona la acción que proporcionará el mayor valor para un estado determinado.

Slide 17

En consecuencia, independientemente del método que utilices para resolver tu problema, tendrás una política.

Solo que en el caso de los métodos basados en valores, no entrenas la política: tu política es simplemente una función predefinida (por ejemplo, la Política Greedy) que utiliza los valores proporcionados por la función de valor para seleccionar sus acciones.

Slide 18

Entonces, la diferencia es:

- En el entrenamiento basado en políticas, la política óptima (denotada como π^*) se encuentra entrenando directamente la política.
- En el entrenamiento basado en valores, encontrar una función de valor óptima (denotada como Q^* o V^*) conduce a obtener una política óptima.

Slide 19

The link between Value and Policy:

$$\pi^*(s) = \arg \max_a Q^*(s, a)$$

Finding an optimal value function leads to having an optimal policy.

Figure 7 Link value and policy

Slide 20

La función de valor de estado

Escribimos la función de valor de estado bajo una política π de la siguiente manera:

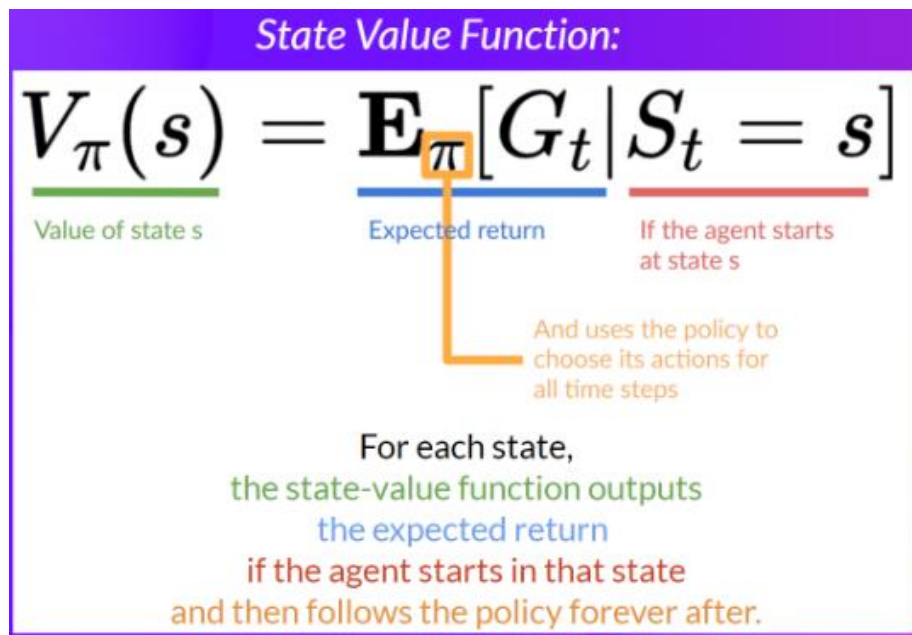


Figure 8 State value function

Slide 21

Para cada estado, la función estado-valor devuelve la rentabilidad esperada si el agente parte de ese estado y, a partir de ahí, sigue la política de forma indefinida (en todos los pasos temporales futuros, si lo prefieres).

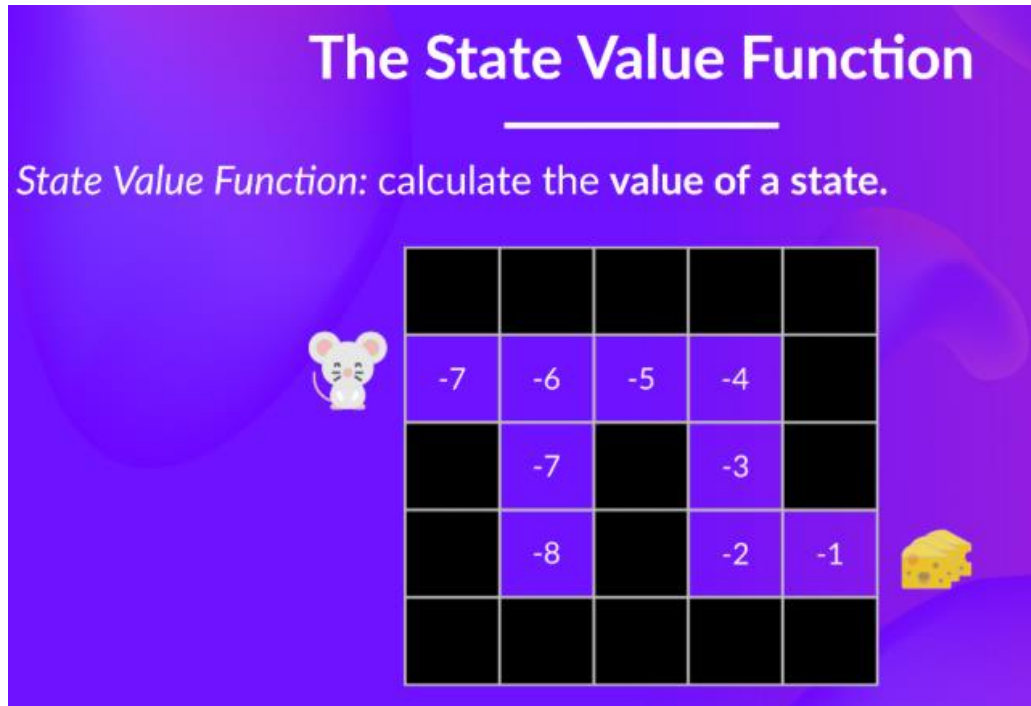


Figure 9 Example state value function

Slide 22

La función de valor de acción

En la función de valor de acción, para cada par de estado y acción, la función de valor de acción devuelve el retorno esperado si el agente comienza en ese estado, toma esa acción y, a partir de entonces, sigue la política para siempre.

Slide 23

El valor de tomar la acción a en el estado s bajo una política π es:

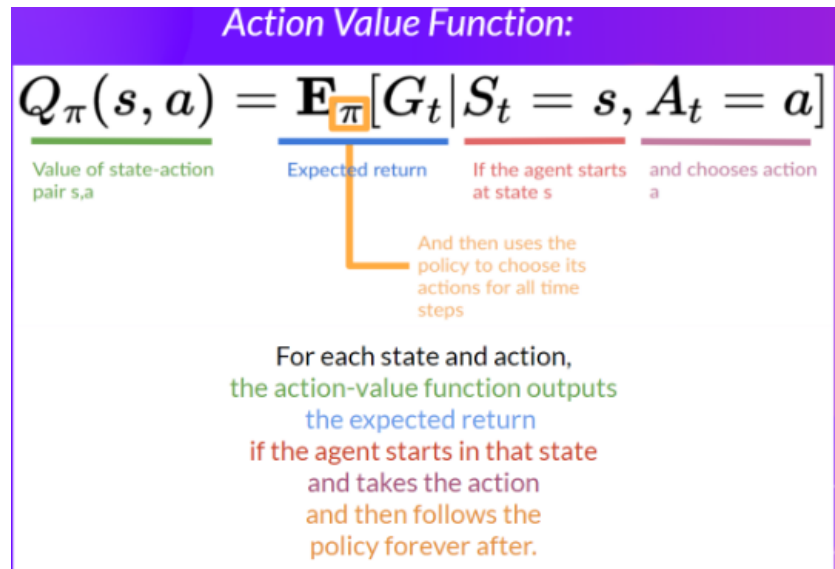


Figure 10 Action value function

Slide 24

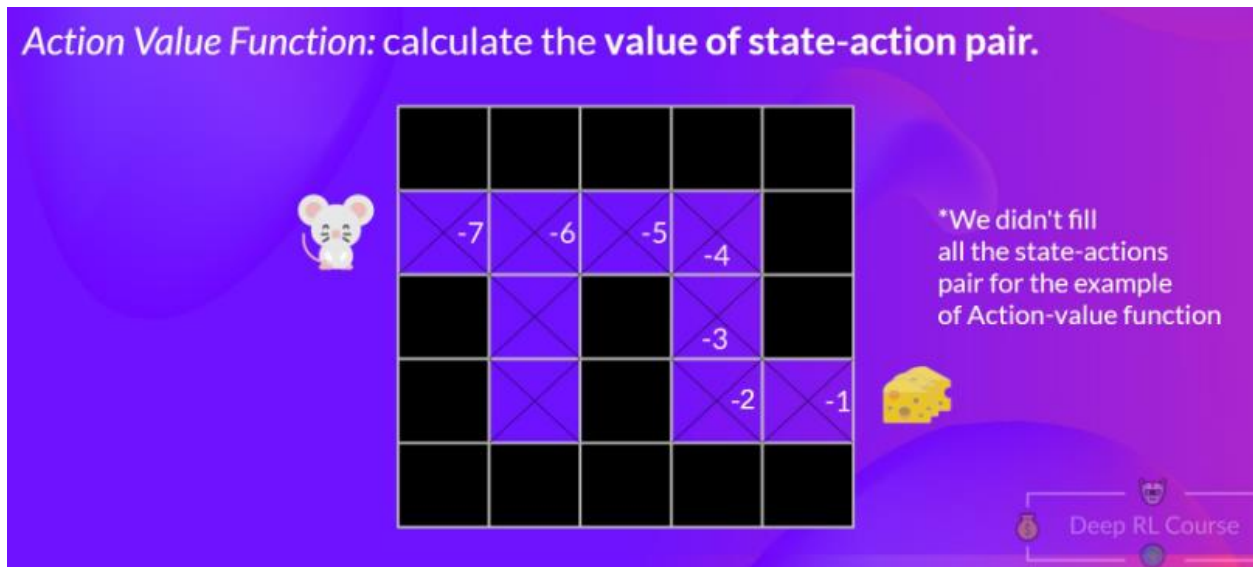


Figure 11 value of state-action pair

Slide 25

Para la función de valor de estado, calculamos el valor de un estado S_t .

Para la función de valor de acción, calculamos el valor del par estado-acción (S_t, A_t) , es decir, el valor de tomar esa acción en ese estado

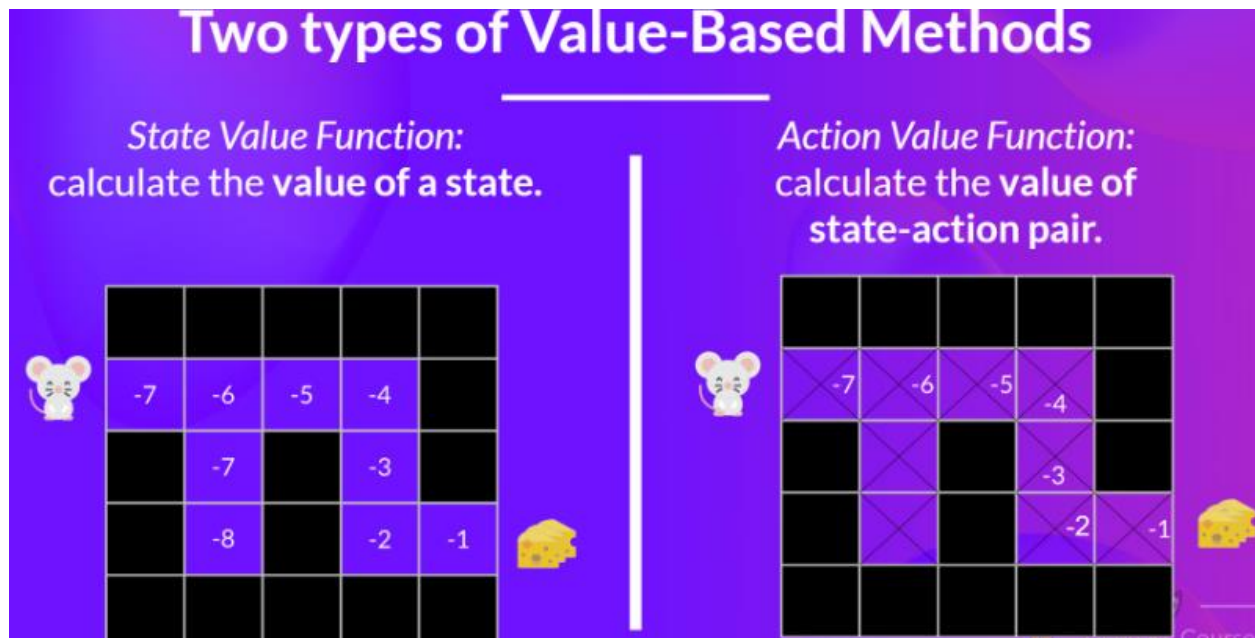


Figure 12 Value-based methods comparisson

Slide 26

El problema es que para calcular cada valor de un estado o de un par estado-acción, necesitamos sumar todas las recompensas que un agente puede obtener si comienza en ese estado.

Este puede ser un proceso computacionalmente costoso, y ahí es donde la ecuación de Bellman entra en juego para ayudarnos.

Slide 27

The Bellman Equation: simplify our value estimation

The Bellman equation simplifies our state value or state-action value calculation.

Slide 28

Sabemos que si calculamos $V(S_t)$ (el valor de un estado), necesitamos calcular el retorno comenzando desde ese estado y luego seguir la política para siempre.

para calcular $V(S_t)$, necesitamos calcular la suma de las recompensas esperadas. Por lo tanto:

Slide 29

Para calcular el valor del Estado 1: la suma de las recompensas si el agente comenzara en ese estado y luego siguiera la política greedy durante todos los pasos de tiempo.

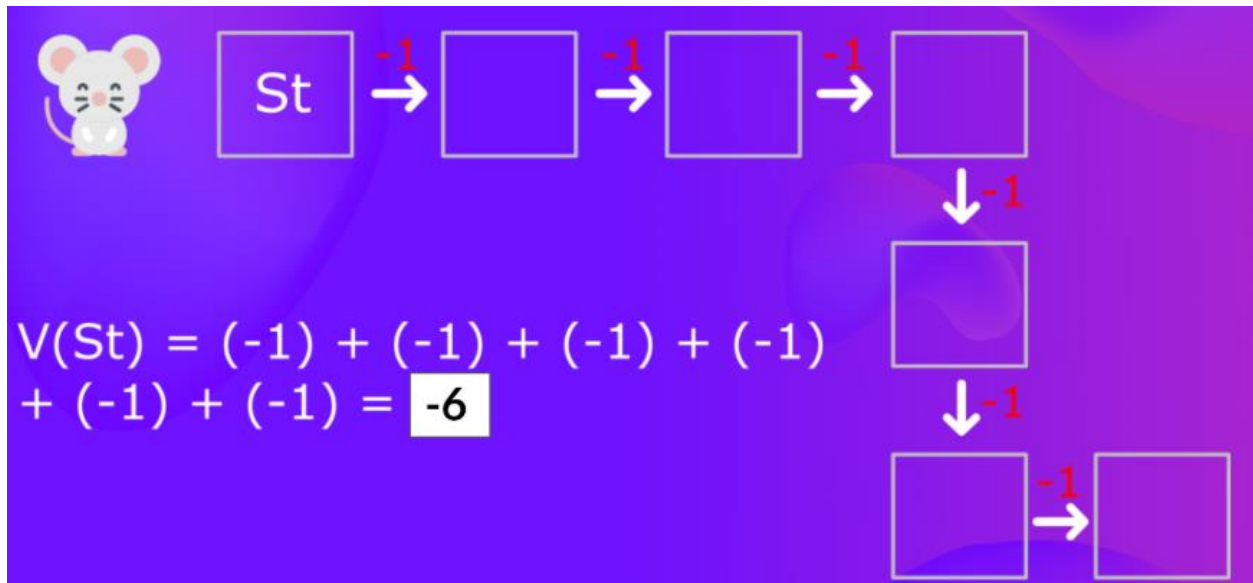


Figure 13 Vst

Slide 30

Luego, para calcular $V(St+1)$, necesitamos calcular el retorno comenzando desde ese estado $St+1$.

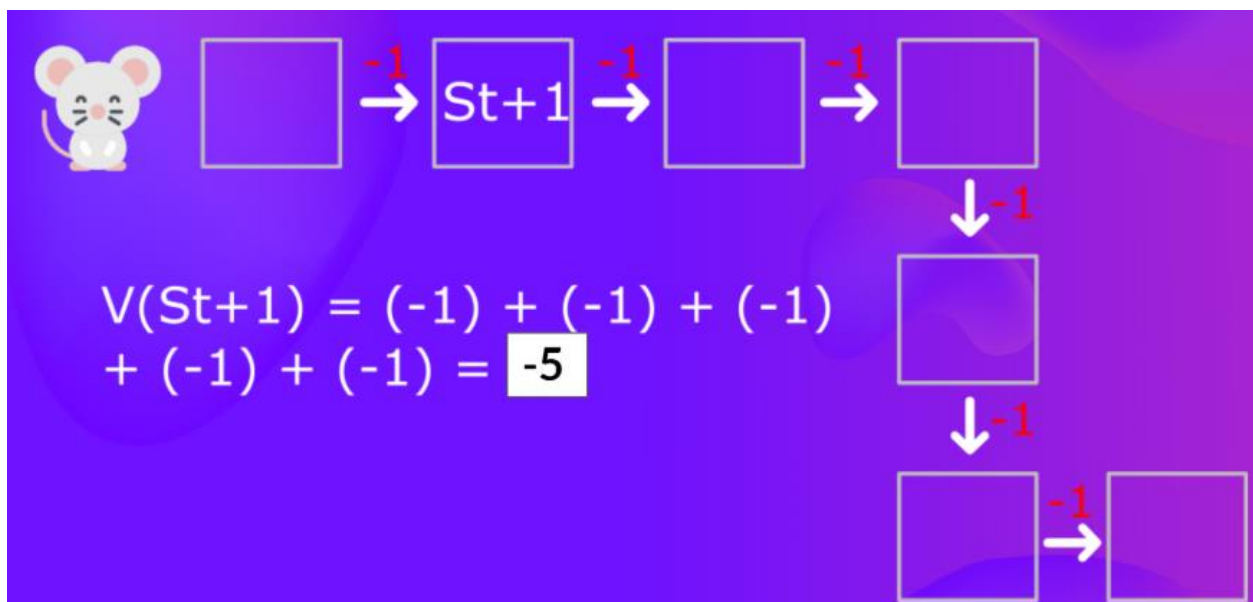


Figure 14 Vst1

Slide 31

Estamos repitiendo el cálculo del valor de diferentes estados, lo cual puede resultar tedioso si necesitamos hacerlo para cada valor de estado o valor de estado-acción.

En lugar de calcular el retorno esperado para cada estado o para cada par estado-acción, podemos utilizar **la ecuación de Bellman**.

Slide 32

ECUACION DE BELLMAN

La ecuación de Bellman es una ecuación recursiva que funciona de la siguiente manera: en lugar de comenzar desde el principio para cada estado y calcular el retorno, podemos considerar el valor de cualquier estado como:

La recompensa inmediata + el valor descontado del estado siguiente:

$$R_{t+1} + (\gamma * V(S_{t+1}))$$

Slide 33

The Bellman Equation

$$V_{\pi}(s) = \mathbf{E}_{\pi} [R_{t+1} + \gamma * V_{\pi}(S_{t+1}) | S_t = s]$$

Value of state s **Expected value of immediate reward** **+ the discounted value of next_state** **If the agent starts at state s**

And uses the policy to choose its actions for all time steps

$V(St) = R_{t+1} + \gamma * V(St+1)$

Deep RL Course

Figure 15 Bellman eq

Slide 34

Para simplificar, aquí no aplicamos descuento, por lo que $\gamma = 1$.

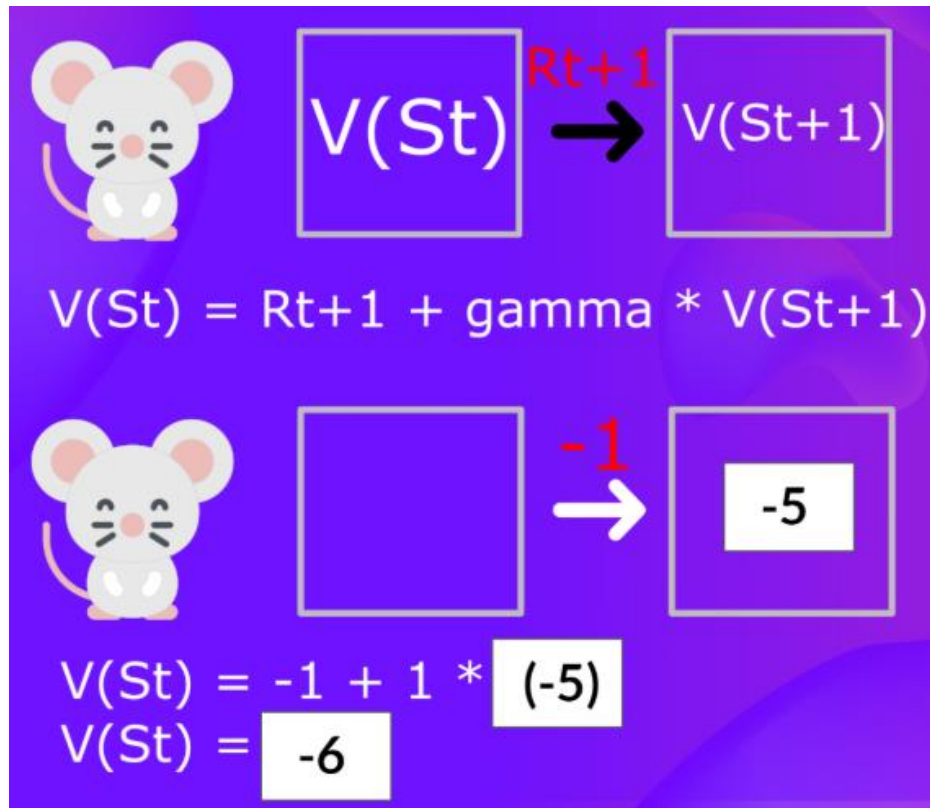


Figure 16 Bellman demo

Slide 35

En resumen, la idea de la ecuación de Bellman es que, en lugar de calcular cada valor como la suma del retorno esperado, lo cual es un proceso largo, calculamos el valor como la suma de la recompensa inmediata + el valor descontado del estado siguiente.

Slide 36

Q-Learning

Método basado en valores y off-policy para entrenar su función de valor de acción:

- *Off-policy*: hablaremos de esto al final de esta unidad.
- *Método basado en valores*: encuentra la política óptima de manera indirecta, entrenando una función de valor o una función de valor de acción que nos indicará el valor de cada estado o de cada par estado-acción.

Slide 37

Q-learning

Actualiza su función de valor de acción en cada paso en lugar de hacerlo al final del episodio.

Q-Learning es el algoritmo que utilizamos para entrenar nuestra función Q, una función de valor de acción que determina el valor de estar en un estado determinado y tomar una acción específica en ese estado.

Slide 38

Dado un estado y una acción, nuestra función Q devuelve un valor de estado-acción (también llamado valor Q).

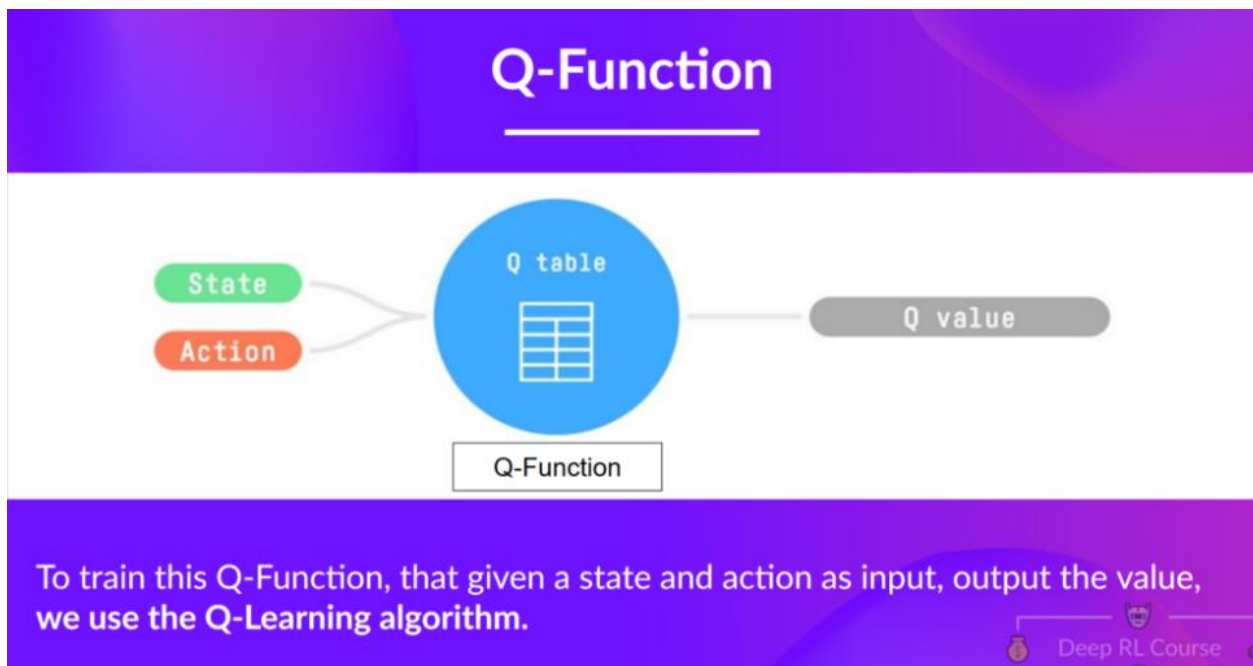


Figure 17 Q function

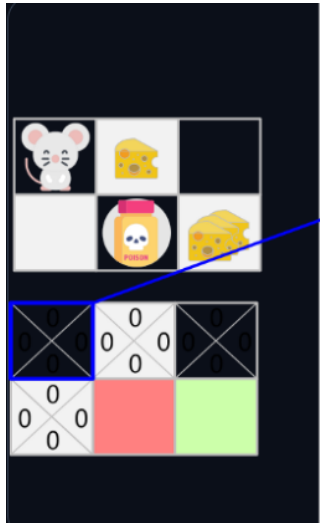
Slide 39

Repasemos la diferencia entre valor y recompensa:

- El *valor de un estado*, o de un *par estado-acción*, es la recompensa acumulada esperada que obtiene nuestro agente si comienza en ese estado (o par estado-acción) y luego actúa de acuerdo con su política.
- La *recompensa* es la retroalimentación que recibe del entorno después de realizar una acción en un estado.

Internamente, nuestra función Q está representada mediante una tabla Q, una tabla en la que cada celda corresponde al valor de un par estado-acción.

Slide 40



		Actions			
		←	→	↑	↓
States		0	0	0	0
		0	0	0	0
		0	0	0	0
		0	0	0	0
		0	0	0	0
		0	0	0	0

Figure 18 Q learning init

La tabla Q está inicializada. Por eso, todos los valores son = 0

Slide 41

Esta tabla contiene, para cada estado y acción, los valores de estado-acción correspondientes. En este ejemplo sencillo, el estado está definido únicamente por la posición del ratón.

Por lo tanto, tenemos 2*3 filas en nuestra tabla Q, una fila por cada posición posible del ratón. En escenarios más complejos, el estado podría contener más información que la posición del agente.

Slide 42

Q-Learning es el algoritmo de RL que:

- Entrena una *función Q* (una función de valor de acción), que internamente es una tabla Q que contiene los valores de todos los pares estado-acción.
- Dado un estado y una acción, nuestra función Q buscará en su tabla Q el valor correspondiente.

- Cuando termina el entrenamiento, tenemos una función Q óptima, lo que significa que tenemos una tabla Q óptima.
- Y si tenemos una función Q óptima, tenemos una política óptima, ya que sabemos cuál es la mejor acción que tomar en cada estado.

Slide 43

Con el entrenamiento, nuestra tabla Q mejora, ya que gracias a ella podemos conocer el valor de cada par estado-acción.



Figure 19 Q learning example

Slide 44

Q-Learning

Algorithm 14: Sarsamax (Q-Learning)

Input: policy π , positive integer $num_episodes$, small positive fraction α , GLIE $\{\epsilon_i\}$
Output: value function Q ($\approx q_\pi$ if $num_episodes$ is large enough)
Initialize Q arbitrarily (e.g., $Q(s, a) = 0$ for all $s \in \mathcal{S}$ and $a \in \mathcal{A}(s)$, and $Q(\text{terminal-state}, \cdot) = 0$)
for $i \leftarrow 1$ **to** $num_episodes$ **do** ↖ Step 1
 $\epsilon \leftarrow \epsilon_i$
 Observe S_0
 $t \leftarrow 0$
 repeat
 Choose action A_t using policy derived from Q (e.g., ϵ -greedy) Step 2
 Take action A_t and observe R_{t+1}, S_{t+1} Step 3
 $Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha(R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t))$ Step 4
 $t \leftarrow t + 1$
 until S_t is terminal;
end
return Q

Figure 20 Algorithm q learning

Slide 45

Q-Learning, Step 1

Initialize Q arbitrarily (e.g., $Q(s, a) = 0$ for all $s \in \mathcal{S}$ and $a \in \mathcal{A}(s)$, and $Q(\text{terminal-state}, \cdot) = 0$)

	←	→	↑	↓
	0	0	0	0
	0	0	0	0
	0	0	0	0
	0	0	0	0
	0	0	0	0
	0	0	0	0

We initialize the Q-Table

Figure 21 Q learning step1

Slide 46

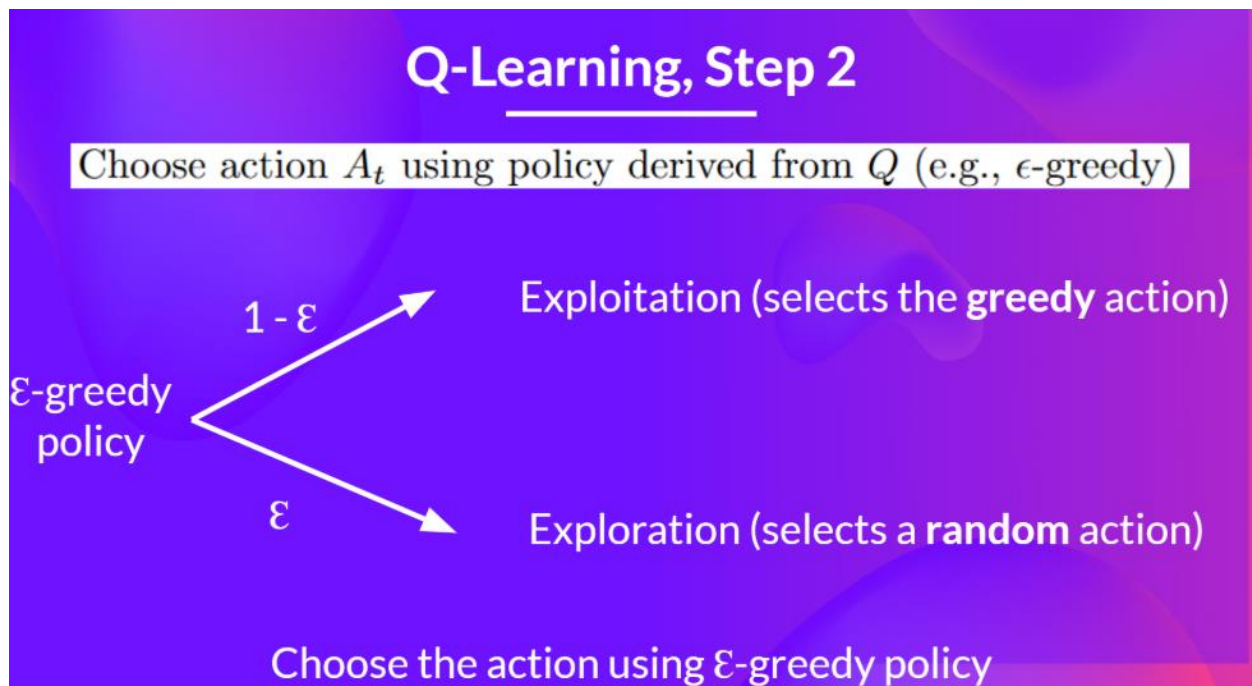


Figure 22 Q learning step2

Slide 47

La estrategia epsilon-greedy es una política que gestiona el equilibrio entre exploración y explotación.

La idea es que, con un valor inicial de $\epsilon = 1.0$:

- Con una probabilidad de $1 - \epsilon$: hacemos **explotación** (es decir, nuestro agente selecciona la acción con el valor más alto para el par estado-acción).
- Con una probabilidad de ϵ : **hacemos exploración** (probando una acción aleatoria).

Slide 48

Al comienzo del entrenamiento, la **probabilidad de realizar exploración** será muy alta, ya que ϵ es muy alto, por lo que la mayor parte del tiempo exploraremos.

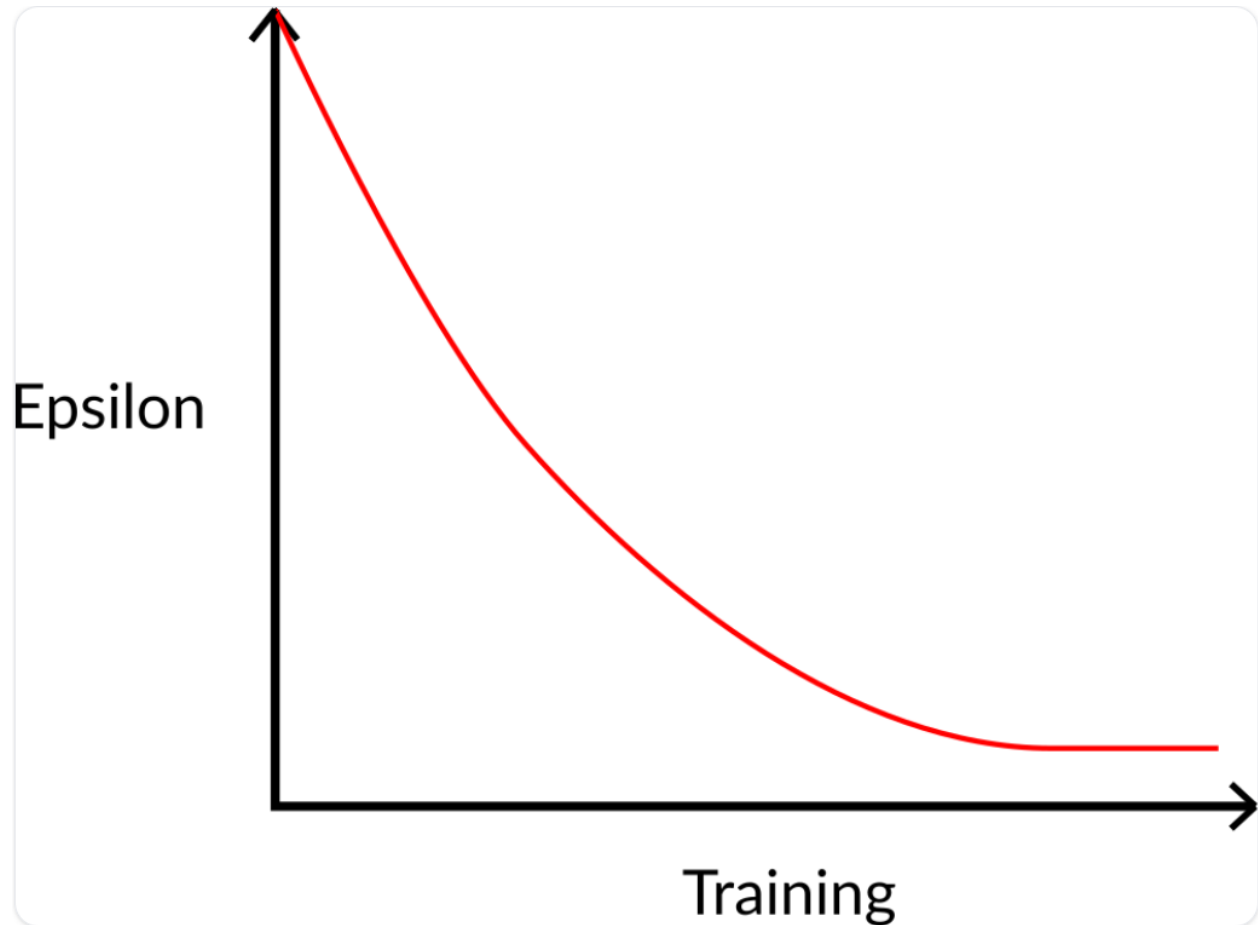


Figure 23 Epsilon in time

Slide 49

Perform action A_t , get reward R_{t+1} and next state S_{t+1}

Q-Learning, Step 3

Take action A_t and observe R_{t+1}, S_{t+1}

Figure 24 Q learning step3

Slide 50

Step 4: Update Q(St, At)

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha[R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t)]$$

New
Q-value
estimation

Former
Q-value
estimation

Learning
Rate

Immediate
Reward

Discounted Estimate
optimal Q-value
of next state

Former
Q-value
estimation

TD Target

TD Error

Figure 25 Q learning Step4